DENTALFLOW ENTERPRISE ARCHITECTURE

Unified Dental Healthcare Platform

Shared Patient Intelligence Architecture & Implementation Roadmap

An exhaustive architectural blueprint unifying deployed DentalFlow practice management and patient communication with an AI-powered X-ray condition detection pipeline through a single canonical patient intelligence foundation.

PREPARED FOR

Mohona (Product Owner)

AUTHOR ROLES

Lead Architect, Healthcare Tech & AI Engineer

DOCUMENT VERSION

1.0.0-PROD-PLAN (Verified)

CLASSIFICATION

Internal Engineering & Clinical Governance

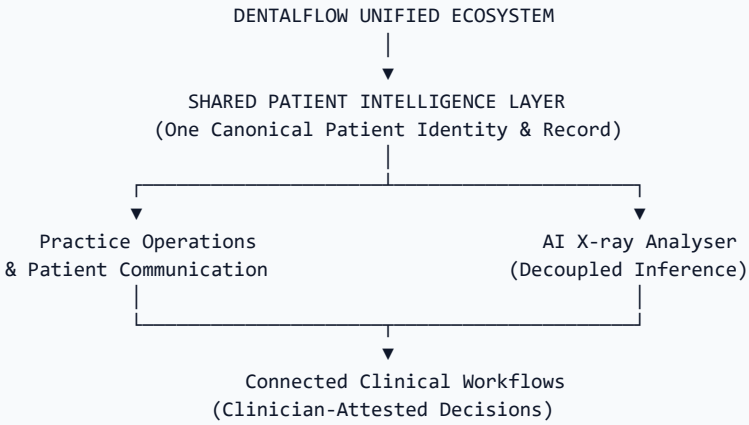
Start with dentistry. Build for healthcare. Scale globally.

October 2026 • Confidential

1. Executive Summary & Product Direction

DentalFlow is evolving from an operational dental clinic and patient communication tool into an integrated healthcare platform. This evolution unifies two existing codebases:

- **Existing DentalFlow Platform**: Practice management, patient self-registration, appointment scheduling, AI receptionist call logs, follow-up automation, analytics, and a 3-section patient portal.
- **AI X-ray Analyser (Dobbe)**: Deep learning detection of dental caries and periapical lesions from standard and DICOM radiographs using Roboflow computer vision models.
- **Shared Patient Intelligence Layer**: The unified canonical patient foundation connecting all modules to a single, immutable patient identity, cross-practice tenancy boundaries, and clinical workflows.



> ⚠️ **Mandatory Scope Decision (Architecture PDF Alignment)**
>
> The **Consultation Analyser has been permanently removed** from the product roadmap. The platform will NOT record, listen to, or synthesize ambient dentist-patient conversations into automated treatment reports due to speech-recognition error risks, background clinic noise, and generative hallucination liabilities. Standard clinician-entered consultation notes remain fully supported as structured EHR records.

2. Source Code Audit of Repositories

2.1 Existing DentalFlow Platform (`Mghosh2005/dentalflow`)

- **Frontend Architecture**: Built with React 19.2.8 and Vite 8.2.2. Uses Tailwind CSS v3.4.19, Lucide React icons, and Recharts. Navigation is managed client-side in `App.jsx` via state switching. Includes a dedicated 3-section Patient Portal (*My Portal*, *My Appointments*, *My Treatment Plan*) and a full staff management suite (Dashboard, Appointments, Patients, Inbox, Follow-ups, Alerts, Practitioners, Analytics, AI Settings).
- **Backend Architecture**: Node.js Express.js application in `server.js` (2,533 lines) using `better-sqlite3` in WAL mode (18 tables). Tables include `practices`, `users`, `user_accounts`, `practitioners`, `patients`, `appointments`, `enquiries`, `follow_ups`, `follow_up_messages`, `message_templates`, `alerts`, `call_logs`, and `treatment_plans`.

- **Auth & Tenancy**: JWT authentication with persistent secrets, rate-limiting (5 failed attempts = 15m lockout), strict role separation (`staff` vs `patient`). Currently hardcoded to single practice (`PRACTICE_ID = 1`).
- **Storage Audit**: **Zero persistent image/file storage currently exists**. Audio transcription takes JSON base64 payloads, creates temporary files, spawns Whisper, and deletes them immediately.
- **Deployment**: Configured for Render Web Service (`render.yaml`). Express serves prebuilt static assets from `frontend/dist` .

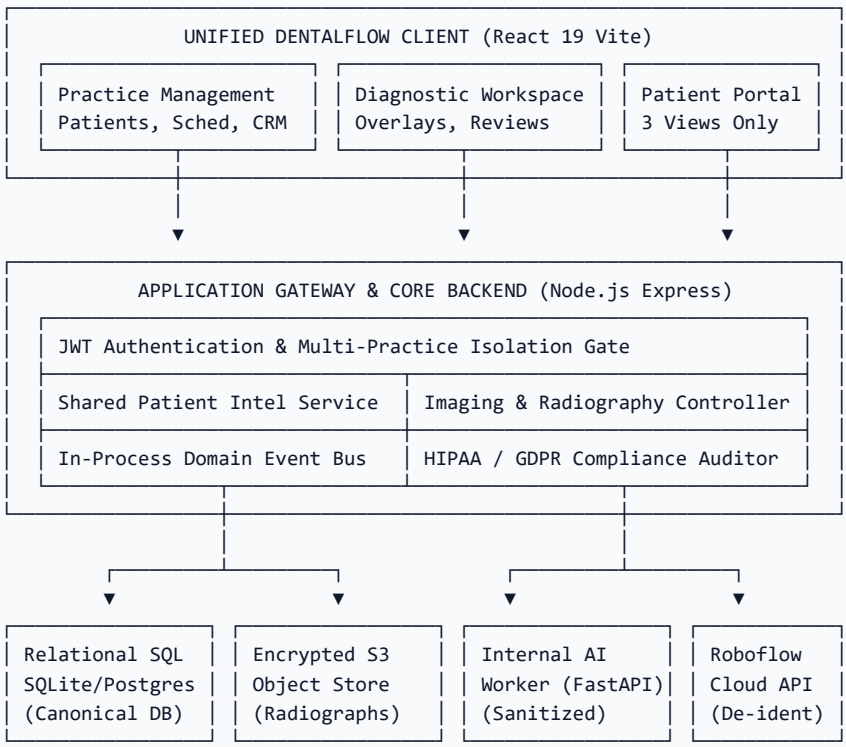
2.2 AI X-ray Analyser (`adityanandanx/dental-conditions-detection` — "Dobbe")

- **Frontend Architecture**: Next.js 15.3.3 App Router, React 19, TypeScript, Tailwind CSS v4, shadcn/ui, Zustand, TanStack Query. Uses `dcmjs-imaging` for WASM DICOM previewing and `react-dropzone` .
- **Backend Architecture**: FastAPI 0.115.12, Python 3.12+, managed via UV. Uses `pydicom` , `gdcm` 1.1 for DICOM decompression, Pillow + NumPy for windowing and PNG conversion.
- **AI Inference Pipeline**: Roboflow Serverless Inference SDK invoking model `adr/6` . Detects `cavity` and `periapical_lesion` with normalized bounding boxes and confidence scores.
- **Report Pipeline**: LangChain + OpenAI `gpt-4o-mini` for narrative report generation.
- **Persistence Reality**: **Completely stateless**. Uploads are handled in temporary files and discarded. No database, no user accounts, no patient linking, and no audit trails exist.
- **Clinical Defect**: The existing pipeline automatically synthesizes clinical diagnoses and treatment suggestions with GPT-4o-mini without clinician-in-the-loop validation, violating medical device standards.

3. Current vs. Target Architecture

3.1 Target Unified Architecture

The target platform deploys the Next.js viewing capabilities directly into the DentalFlow React frontend, while the Python FastAPI service is transformed into a private, decoupled AI worker.



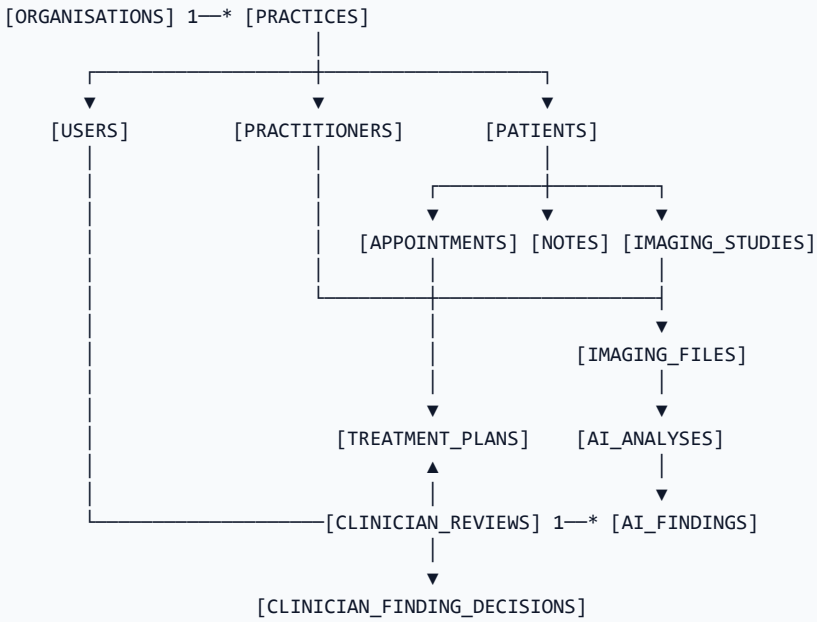
4. Feature and Module Integration Map

Capability	Existing Source	Target Unified Location	Integration Mechanism
Patient Demographics & Identity	DentalFlow (`patients`)	Shared Patient Intelligence Layer	Extended with `canonical_uuid`, MRN, structured allergies.
Appointments & Scheduling	DentalFlow (`appointments`)	Practice Operations Module	Preserved; references canonical Patient ID.
Patient Portal (3 Sections)	DentalFlow (`pages/patient/`)	Patient Portal Presentation Layer	Preserved intact; only renders clinician-attested treatment plans.
Automated Follow-ups	DentalFlow (`follow_up_messages`)	Practice Operations Module	Preserved; triggered by operational domain events.
DICOM Parsing & Ingestion	Dobbe (`inference_service.py`)	Internal AI Imaging Subsystem	Ingested via private worker; metadata persisted in core DB.
AI Condition Detection (adr/6)	Dobbe (Roboflow Client)	Internal AI Imaging Subsystem	Decoupled async worker; returns structured unconfirmed findings.
Medical Viewer Canvas	Dobbe (`components/results/`)	Unified Frontend (`ImagingWorkspace`)	Integrated into patient profile and practice review queue.
Clinician Review & Attestation	**None (New)**	Shared Patient Intelligence Layer	Formal medical attestation drawer with 1-click treatment promotion.

5. Canonical Patient Data Model & Schema

To ensure zero data loss, all existing DentalFlow tables are preserved. Extensions are strictly additive using nullable columns or safe defaults.

5.1 Entity Relationship Diagram (ERD)



5.2 Core Schema Additions (SQL DDL)

```
-- 1. Canonical Patient Identity Extensions
ALTER TABLE patients ADD COLUMN canonical_uuid TEXT;
ALTER TABLE patients ADD COLUMN medical_record_number TEXT;
ALTER TABLE patients ADD COLUMN allergies_json TEXT DEFAULT '[]';
ALTER TABLE patients ADD COLUMN medical_alerts_json TEXT DEFAULT '[]';
ALTER TABLE patients ADD COLUMN matching_status TEXT DEFAULT 'verified';
ALTER TABLE patients ADD COLUMN merged_into_patient_id INTEGER REFERENCES patients(id);

CREATE UNIQUE INDEX IF NOT EXISTS idx_patients_uuid ON patients(canonical_uuid);
CREATE UNIQUE INDEX IF NOT EXISTS idx_patients_mrn ON patients(practice_id, medical_record_number);

-- 2. Clinician-Entered Consultation & Encounter Notes
CREATE TABLE IF NOT EXISTS clinical_notes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  uuid TEXT UNIQUE NOT NULL,
  practice_id INTEGER NOT NULL REFERENCES practices(id),
  patient_id INTEGER NOT NULL REFERENCES patients(id),
  practitioner_id INTEGER NOT NULL REFERENCES practitioners(id),
  appointment_id INTEGER REFERENCES appointments(id),
  note_type TEXT DEFAULT 'general_encounter',
  content TEXT NOT NULL,
  signed_by_practitioner_id INTEGER REFERENCES practitioners(id),
  signed_at TEXT,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- 3. Imaging Studies & Physical Radiographic Files
CREATE TABLE IF NOT EXISTS imaging_studies (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  uuid TEXT UNIQUE NOT NULL,
  practice_id INTEGER NOT NULL REFERENCES practices(id),
  patient_id INTEGER NOT NULL REFERENCES patients(id),
  appointment_id INTEGER REFERENCES appointments(id),
  study_instance_uid TEXT,
  study_date TEXT NOT NULL,
  modality TEXT NOT NULL, -- IO (Intraoral), PX (Panoramic), DX
  study_description TEXT,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS imaging_files (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  uuid TEXT UNIQUE NOT NULL,
  study_id INTEGER NOT NULL REFERENCES imaging_studies(id),
  practice_id INTEGER NOT NULL REFERENCES practices(id),
  patient_id INTEGER NOT NULL REFERENCES patients(id),
  file_storage_key TEXT NOT NULL,
  original_filename TEXT NOT NULL,
  file_format TEXT NOT NULL, -- dicom, png, jpeg
  file_size_bytes INTEGER NOT NULL,
  sha256_hash TEXT NOT NULL,
  pixel_width INTEGER,
  pixel_height INTEGER,
  dicom_metadata_json TEXT,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 4. AI Inference Executions & Candidate Findings
CREATE TABLE IF NOT EXISTS ai_analyses (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  uuid TEXT UNIQUE NOT NULL,
  imaging_file_id INTEGER NOT NULL REFERENCES imaging_files(id),
  practice_id INTEGER NOT NULL REFERENCES practices(id),
  patient_id INTEGER NOT NULL REFERENCES patients(id),
  model_identifier TEXT NOT NULL, -- adr/6
  model_version TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'queued', -- queued, processing, completed, failed
  raw_inference_json TEXT,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS ai_findings (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  analysis_id INTEGER NOT NULL REFERENCES ai_analyses(id),
  condition_label TEXT NOT NULL, -- cavity, periapical_lesion
  confidence_score REAL NOT NULL,
  bbox_x REAL NOT NULL, bbox_y REAL NOT NULL,
  bbox_width REAL NOT NULL, bbox_height REAL NOT NULL,
  tooth_number_fdi TEXT,
  initial_status TEXT DEFAULT 'potential'
);

-- 5. Clinician Attestation & Itemized Decisions
CREATE TABLE IF NOT EXISTS clinician_reviews (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  uuid TEXT UNIQUE NOT NULL,
  analysis_id INTEGER NOT NULL REFERENCES ai_analyses(id),
  practice_id INTEGER NOT NULL REFERENCES practices(id),
  patient_id INTEGER NOT NULL REFERENCES patients(id),
  reviewer_user_id INTEGER NOT NULL REFERENCES user_accounts(id),
  review_status TEXT NOT NULL, -- accepted_as_is, accepted_with_edits, rejected_false_positive
  clinical_notes TEXT,
  reviewed_at TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS clinician_finding_decisions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  review_id INTEGER NOT NULL REFERENCES clinician_reviews(id),
  finding_id INTEGER NOT NULL REFERENCES ai_findings(id),
  decision TEXT NOT NULL, -- confirmed, rejected, adjusted
  confirmed_condition TEXT,
  treatment_recommended TEXT
);
```

6. Safe Patient-Matching and Duplicate Prevention

> 🚨 **Patient Safety Hazard: Name-Only Matching Prohibited**
>
> Associating radiographs with a patient based solely on a patient name match represents a catastrophic clinical safety risk. DentalFlow strictly prohibits single-attribute name matching.

```
MATCHING PROTOCOL FLOWCHART
[Incoming Radiograph File]
         │
         ▼
[Direct Patient Context?] ─── YES ──► Verify Tenant Ownership ──► Link to Patient ID
         │ (NO)
         ▼
[Deterministic Match?]   ─── YES ──► Check (MRN) OR (National Health ID) ──► Link
         │ (NO)
         ▼
[Compound Match?]        ─── YES ──► Exact DOB + (Exact Phone OR Email) ──► Link
         │ (NO)
         ▼
[Ambiguous or Fuzzy]     ──────────► QUARANTINE QUEUE ──► Staff Manual Review & Approval
```

7. API Specifications & Event Architecture

7.1 Versioned API Contracts

HTTP Endpoint	Method	Auth Role	Description & Safety Controls
`/api/v1/patients/:id/timeline`	`GET`	Staff / Patient	Returns longitudinal record across visits, notes, imaging, and care plans.
`/api/v1/imaging/upload`	`POST`	Staff	Multipart upload (DICOM/PNG/JPEG); SHA256 integrity check; AES-256 storage.
`/api/v1/imaging/files/:id/analyze`	`POST`	Staff	Asynchronously dispatches de-identified pixel buffer to AI worker.
`/api/v1/imaging/analyses/:id/review`	`POST`	Practitioner	**Formal Clinician Attestation**: Accepts/edits/rejects findings; optional 1-click push to treatment plans.

7.2 Asynchronous Domain Event Taxonomy

Event Name	Trigger Event	Technical Semantics	Clinical Semantics
`XrayUploaded`	Radiograph stored	File integrity verified	Image ingested into patient study.
`XrayAnalysisCompleted`	Inference finishes	Bounding boxes calculated	**Technical event only — ZERO clinical confirmation.**
`AIFindingReviewed`	Clinician signs off	Decisions committed	**Legally binding clinical attestation.**
`TreatmentPlanUpdated`	Care plan modified	Procedure recorded	Active clinical regimen updated.

8. Clinical Safety & Regulatory Governance

8.1 Separation of Potential vs. Confirmed Findings

- **Unconfirmed Status Invariant**: Machine predictions are immutably labeled as `potential` until a licensed clinician submits an attestation.
- **Zero Automatic Treatment Generation**: Under no circumstances does an unreviewed AI detection insert a procedure into `treatment_plans` or dispatch communications to a patient.
- **Complete Cryptographic Provenance**: Every finding in a patient chart permanently records: `Image SHA-256` $ightarrow$ `Model Version (adr/6)` $ightarrow$ `Raw Bounding Box Coordinates` $ightarrow$ `Attesting Clinician ID & Timestamp`.

8.2 HIPAA / GDPR Privacy Boundary

- **DICOM De-identification**: Before any image is processed by third-party vision engines (Roboflow), all DICOM header PHI (Name, DOB, MRN, Clinic Details) is stripped. The external API receives only raw pixel buffers mapped to an ephemeral UUID.
- **Tenant Data Isolation**: Parameterized queries enforce `WHERE practice_id = ?` on all data access. Cross-practice leakage is strictly blocked.
- **Audit Trails**: Read and write operations on radiographic studies generate immutable records in `audit_events`.

9. User Experience & Navigation Architecture

The user experience is consolidated into a single application to eliminate disjointed portal switching:

- **Patient Directory & Profile (Extended)**: `Patients.jsx` receives a new "Imaging & Radiographs" tab displaying historical studies with review badges (`Pending Review`, `Confirmed`, `Normal`) and direct upload triggers.
- **Clinical Imaging Workspace (New Page)**: Houses practice-wide unreviewed radiograph queues and an interactive medical image viewer supporting pan, zoom, contrast windowing, condition overlay toggles, confidence filtering, and a clinician review drawer.
- **Patient Portal (Preserved Intact)**: Remains strictly partitioned into 3 sections (*My Portal*, *My Appointments*, *My Treatment Plan*). Renders only clinician-attested procedures.

10. Phased Implementation Roadmap

Phase	Milestone Scope	Key Deliverables	Exit Gate Criteria
Phase 1	Audit & Specification	Completed technical audit & master architecture plan	Plan approval by product owner
Phase 2	Shared Patient Model	Additive migrations in `db.js`; UUID & MRN backfill; `patient-service.js`	Zero existing patient data loss; unique UUIDs
Phase 3	Service & Event Foundation	`event-bus.js`; timeline endpoint (`/api/v1/patients/:id/timeline`)	Chronological patient timeline verified
Phase 4	Reconnect Existing DentalFlow	Rewire `api/client.js`; verify self-registration & booking	100% pass on `test_verification_suite.mjs`
Phase 5	Secure Image Storage	`multer` validation; encrypted S3 storage; `imaging_files` tables	Genuine DICOM accepted; corrupt files rejected
Phase 6	AI X-ray Analyser Integration	FastAPI worker; async analysis job queue; de-identification pipeline	Worker returns structured potential findings
Phase 7	Clinician Review Workspace	`ImagingWorkspace.jsx`; attestation drawer; 1-click plan promotion	Attestation required before treatment addition
Phase 8	Clinical Validation & Sign-off	Dentist evaluation; sensitivity/specificity benchmarks; HIPAA audit	Formal safety board clearance

11. Recommended First Development Milestone

> 🎯 **Milestone 1: "Shared Patient Intelligence Foundation" (Phases 2 & 3)**
>
> **Primary Deliverable**: Canonical Patient Identity, Schema Backfill, Patient Service, and Timeline API.

Why Milestone 1 Must Precede All Other Work

1. **Prevents Duplicate Identities**: Integrating an image analyzer before establishing a canonical patient model leads to duplicate, fractured patient accounts that require complex manual reconciliation later.
2. **Provides Immediate Operational Value**: Creating the longitudinal timeline enriches existing scheduling, follow-ups, and patient charts immediately, with zero clinical safety risks.
3. **Guarantees Zero Regressions**: Additive migrations ensure existing production operations on Render continue uninterrupted.

Sprint 1 Task Backlog

1. Implement additive migrations in `backend/src/db.js` (`canonical_uuid` , `medical_record_number` , `allergies_json` , `matching_status`).
2. Execute deterministic migration backfill script for existing patient records.
3. Build `backend/src/services/patient-service.js` with multi-tier matching.
4. Implement in-process domain event bus (`backend/src/events/event-bus.js`).
5. Deploy `GET /api/v1/patients/:id/timeline` endpoint.
6. Run regression suite `test_verification_suite.mjs` to verify 100% operational integrity.
